

Lab Work 05

Topic	Creational, Structural, Behavioural Design Patterns
Objective	Implement basic versions of common design patterns to solve recurring design problems in OOP.
AI Copilot Usage	Not Allowed

Tasks

1. Singleton Pattern:

- Imagine a central **SchoolRegistry** class responsible for keeping track of all **Person** objects (students, teachers) created. We only want *one* instance of this registry.
- Implement the **SchoolRegistry** class using the Singleton pattern. A common way in Python is using a class variable to store the instance and overriding `__new__`.
- Add methods like **add_person(self, person)** and **get_person_by_id(self, person_id)** (you might need to decide if student/employee IDs are unique across types or add a common ID).
- Test it: Get the instance multiple times (**reg1 = SchoolRegistry()**, **reg2 = SchoolRegistry()**) and verify they are the same object (**reg1 is reg2**). Add some people and retrieve them.

2. Factory Method Pattern:

- Create a **PersonFactory** class.
- Implement a method **create_person(self, person_type, **kwargs)** inside the factory.
- This method should take a **person_type** string (e.g., "student", "teacher", "undergrad", "postgrad") and keyword arguments (****kwargs**) containing the necessary details (like **student_id**, **first_name**, **last_name**, **major**, etc.).
- Based on the **person_type**, the factory method should instantiate and return the correct subclass of **Person** (**Student**, **Teacher**, **UndergraduateStudent**, etc.). Handle invalid types.
- Test the factory by creating different types of people using the factory method. Add them to your Singleton **SchoolRegistry**.

3. Observer Pattern (Simple Implementation):

- Modify the **Unit** class. Add methods **attach(self, observer)** and **detach(self, observer)** to manage a list of observers.

- b) Add a ***notify(self, message)*** method that iterates through the observers and calls an ***update(message)*** method on each.
 - c) Make the ***Student*** class an observer. Add an ***update(self, message)*** method that simply prints the message received (e.g., "Notification for [Student Name]: [message]").
 - d) Modify the ***Unit.add_student*** method. When the unit becomes full, call ***self.notify("Unit is now full!")***.
 - e) Test: Create a unit with low capacity. Create students. Attach some students as observers to the unit. Enroll students until the unit is full and observe the notification being printed for the attached observers.
4. **Creative Challenge (Optional, if time permits):**
- a) Implement the **Adapter Pattern**. Imagine you have an external ***LegacyHRSystem*** class (you'll need to define a simple mock version) with a method like ***fetch_employee_data(emp_id)*** that returns a dictionary with different key names (e.g., ***{'empID': 'e123', 'name': 'Carol White', 'role': 'Lecturer'}***).
 - b) Create an ***HRAdapter*** class that takes an instance of ***LegacyHRSystem***.
 - c) Implement a method in the adapter, like ***get_teacher_details(self, emp_id)***, that calls the legacy system's method, receives the dictionary, and transforms it into a format suitable for creating your ***Teacher*** object (e.g., extracts first/last name, maps 'Lecturer' to a valid subject if possible, etc.).

Important Notes

- This is an individual task. All code must be your own original work.
- Ask questions and seek help from the lecturer or teaching assistants if you are stuck.
- Focus on writing clean, well-organized, and well-documented code.